rsna-intracranial-aneurysm-detection

6th Place Solution

Rank	6
Team	Ian, Theo & Bartley
Author	Theo Viel
Topic ID	611925
Version	Original

Thanks to the hosts for once again a nice challenge. We always enjoy joining RSNA competitions :)

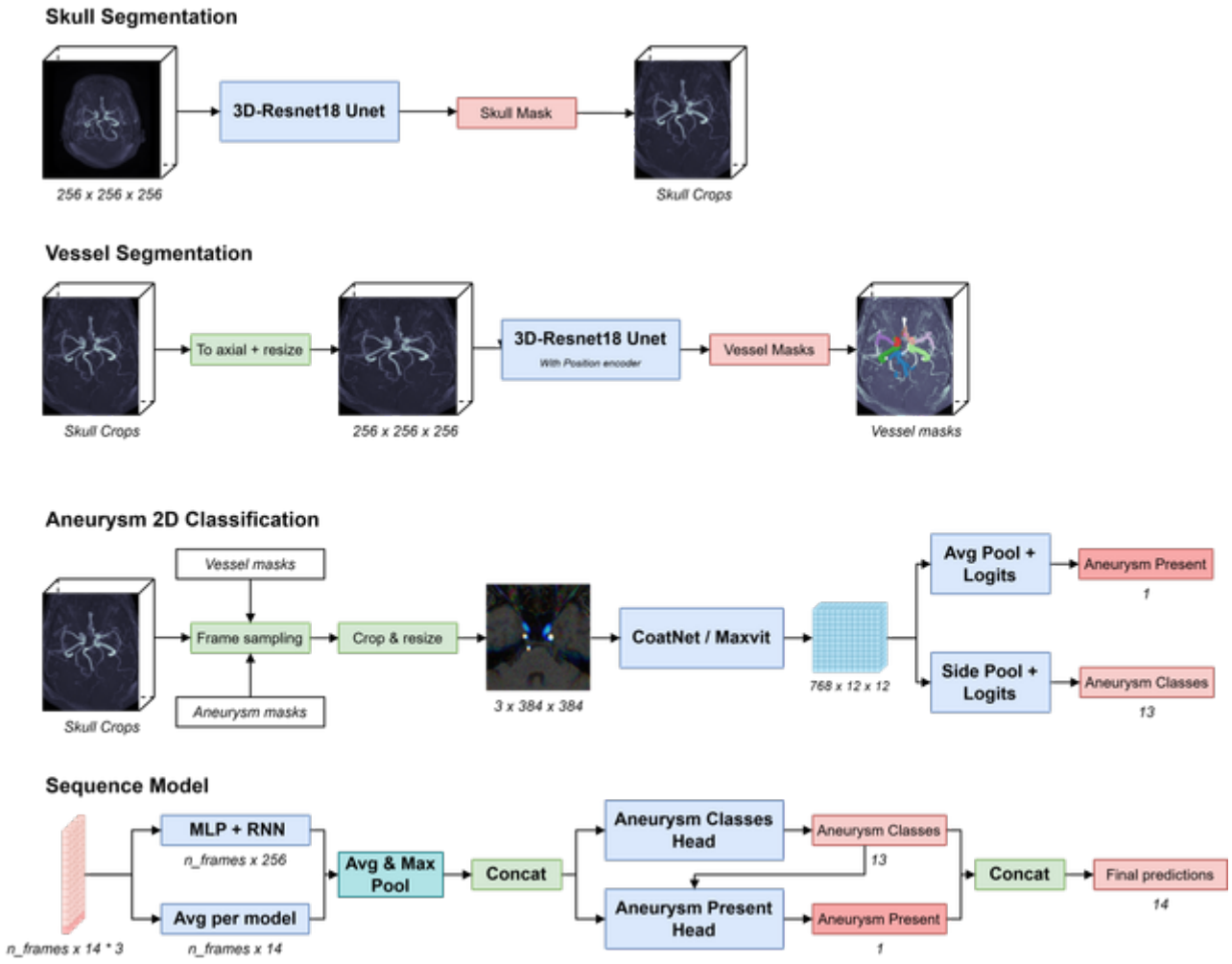
Overview

Our solution is a 2.5D pipeline which consists of 4 steps:

- Skull cropping
- Vessel segmentation (only at train time)
- 2D Aneurysm frame-level classification
- Aggregation using a sequence model

It is inspired by Theo's 2nd place solution from 2 years ago - although the pipeline required a lot of adjustments to achieve decent performance.

It achieves **CV 0.895 - Public 0.84 - Private 0.84**



Pipeline Overview. Click here for the full size the image.

Models

ROI Skull cropping

Nothing fancy, the model is a simple 3D-Unet trained with some open-source data found online. The task is quite easy and the model works on all the orientations. This allows for more uniformity across the dataset.

Vessel segmentation

The task is very similar to the skull segmentation task, but way harder. This model only supports axial data, and requires some tricks to achieve good performance:

- Position encoding to feed left / right information to the CNN
- Mask dilation + dice to fight class imbalance

This model is only used to sample good negatives to feed the 2D models. The initial goal was also to train a vessel crop classification model - but this approach did quite poorly.

2D classification

Main points

This is where most of the heavy lifting comes from. We heavily optimized 2D-CNN models by training them with cleverly sampled frames.

Architectures used are `coatnet_rmlp_2_rw_384` and `maxvit_rmlp_base_rw_384`. The Relative Position Encoding MLP(rmlp) helps a lot since position information is very important to distinguish the 13 classes. On top of it, a custom pooling is used not to lose the left/right information before the logits layer.

This model works on coronal and axial data: sagittal stacks are converted to axial during inference, and ignored during training.

More Ideas

Each of the Following approximately brought around +0.01 CV

- Further cropping the ROI by using fixed ratios to restrict the skull to areas that actually contain aneurysm.
- Strong ShiftScaleRotate and color augmentations
- Cutmix (or Mixup) to prevent overfitting
- Horizontal flip augmentation that also flips the targets
- Use 2 adjacent frames as 3 channels
- Use SliceSpacing information to sample the adjacent frames further if the spacing is small.
- Ian manually refined the localizers labels to obtain segmentation masks. This allowed for more accurate frame sampling for positives

Other things that did not really help CV but were kept for robustness

- External data from OpenNeuro.org
- Axial -> Coronal augmentation

Sequence models

Using a simple max aggregation using predictions of the 2D models on all the frames already gave 0.87 CV. Adding a custom sequence model further improved results to 0.88. This also allowed for easy ensembling, with a 3-model ensemble reaching **0.895 CV**.

To reduce inference time, the maximum number of frames is limited to a fixed number depending on the modality. Furthermore, only half of the frames are inferred for stacks of size > 64, and a quarter for stacks > 128. The model uses 3 adjacent frames as input which means the information is not lost anyways.

Final words

Using 2 models instead of 3 in the pipeline would make the pipeline run in 7 hours, which would probably have been enough to blend-in a 3D pipeline. We also had strong results with such approaches (LB 0.79). However CV improvements were small (<0.01) and due to submission runtime instability and our CV being concerningly high compared to LB, we decided not to invest more time there.

Thanks for reading !

Supplemental Q&A

> **Optimo · 2025-10-15T16:35:00.457000**
>
> Congratulations to the entire team!
> All images are simply minmaxed normalized? No windowing?
>
> Did you try training from scratch the 2.5D model?

> **Theo Viel · 2025-10-15T17:09:28.013000**
>
> Normalization is the following:
>
> ```
> min_ = .percentile(.(), )
> max_ = .percentile(.(), )
>  = .clip(, min_, max_)
>
> eps =   min_ == max_
>  = (.astype(.float32) - min_) / (max_ - min_ + eps)
> ```
>
> No luck with CT windowing, at least for the 2D models.
>
> > Did you try training from scratch the 2.5D model?
>
> Do you mean end-to-end CNN + LSTM head ?

> **Optimo · 2025-10-15T18:27:54.347000**
>
> Yes end to end training

> **Theo Viel · 2025-10-15T21:13:30.733000**

> Multiple frames + RNN head performed quite poorly.
> I did not try to feed all the frames though. Seems quite heavy computationally and I think models will overfit very fast.